

De Verborgen Matrix

Een puzzel in lineaire algebra over $\text{GF}(2)$

*Achter twee matrices van 29 bij 29 nullen en enen
schuilt een derde matrix, verscholen achter vier fasen van berekening.
De enige weg erheen: Gauss-eliminatie, volledig modulo 2.*

Inhoud

1	De opdracht — overzicht	2
2	Fase 1 — Het sleutelgetal	3
3	Fase 2 — Van sleutelgetal naar matrices: het LFSR	4
3.1	Wat is een LFSR?	4
3.2	Volledig uitgewerkt minivoorbeeld (register van lengte 4)	4
3.3	Van bitreeks naar matrices	4
3.4	Invulrooster — matrix A	6
3.5	Invulrooster — maskeermatrix M	7
4	Fase 3 — De vercijferde matrix demaskeren	8
5	Fase 4 — Gauss-Jordan-eliminatie modulo 2	10
5.1	Het principe	10
5.2	Het algoritme, stap voor stap	10
5.3	Volledig uitgewerkt minivoorbeeld (3×3)	10
5.4	Werkblad — de uitgebreide matrix $[A \mid B]$ (29×58)	12
5.5	Invulrooster — eindresultaat X	13

1 De opdracht — overzicht

Deze keer krijg je matrix A niet cadeau. Je moet ze zelf opbouwen. Het traject verloopt in vier fasen:

1. **Los een vergelijking op** om een geheim *sleutelgetal* S te vinden (een getal tussen 0 en $2^{29} - 1$).
2. **Voer S in een schuifregister** (een LFSR — meer hierover in §3) en genereer daarmee, bit voor bit, twee matrices: matrix A zelf, én een *maskeermatrix* M .
3. Je krijgt een **vercijferde matrix** B_{enc} . Met je zelfgemaakte M demaskeer je die tot de echte matrix B .
4. Los tot slot het stelsel $A \cdot X \equiv B \pmod{2}$ op via Gauss-eliminatie. De oplossing X is een unieke 29×29 -matrix van 0'en en 1'en.

2 Fase 1 — Het sleutelgetal

Bereken:

$$S = \frac{2000}{\pi} \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}.$$

Wat je nodig hebt voor de volgende fase. S moet een geheel getal zijn tussen 0 en $2^{29} - 1 = 536\,870\,911$. Zodra je S hebt, zet je het om naar een rij van precies 29 bits (met voorlooppullen indien nodig) — dat wordt de seed voor Fase 2.

3 Fase 2 — Van sleutelgetal naar matrices: het LFSR

3.1 Wat is een LFSR?

Een **lineair-terugkoppelingsschuifregister** (Engels: *linear feedback shift register*, LFSR) is een eenvoudig mechanisme dat een oneindige reeks bits genereert uit een eindig geheugen. Het bestaat uit een rij van d geheugenposities (hier $d = 29$), gevuld met een startwaarde (de *seed*). Bij elke stap:

1. Bereken een **terugkoppelbit**: de XOR van de bits op vaste, vooraf afgesproken posities (de *tap-posities*).
2. Schuif alle bits één positie op.
3. De nieuwe terugkoppelbit komt binnen aan het ene uiteinde; de bit die aan het andere uiteinde “uitvalt” is de **output-bit** van deze stap.

Dit is exact dezelfde soort wiskunde (lineaire recursies modulo 2) als bij stroomvercijfering en foutcorrectie in de cryptografie en codeertheorie — het past dus naadloos bij de rest van deze puzzel.

Voor deze puzzel: register van lengte $d = 29$, tap-posities 29 en 2 (dit hoort bij de *primitieve veelterm* $x^{29} + x^2 + 1$ over $\text{GF}(2)$ — een standaardkeuze uit de codeertheorie die garandeert dat de reeks pas na $2^{29} - 1$ stappen zichzelf herhaalt).

3.2 Volledig uitgewerkt minivoorbeeld (register van lengte 4)

Ter illustratie: een LFSR met $d = 4$, tap-posities 4 en 1, en seed $1001_2 = 9$. Het register wordt weergegeven als (r_1, r_2, r_3, r_4) , met r_1 het meest linkse bit.

stap	register vóór de stap	output-bit (= r_4 vóór schuiven)
1	(1, 0, 0, 1)	1
2	(0, 1, 0, 0)	0
3	(0, 0, 1, 0)	0
4	(0, 0, 0, 1)	1
5	(1, 0, 0, 0)	0
6	(1, 1, 0, 0)	0
7	(1, 1, 1, 0)	0
8	(1, 1, 1, 1)	1

Toelichting bij stap 1: terugkoppelbit = $r_4 \oplus r_1 = 1 \oplus 1 = 0$. De output-bit van deze stap is het bit dat rechts uitvalt: $r_4 = 1$. Na het schuiven wordt het nieuwe register (0, 1, 0, 0) — de terugkoppelbit (0) komt links binnen, en r_1, r_2, r_3 schuiven één plaats op.

Toelichting bij stap 4: register is (0, 0, 0, 1). Terugkoppelbit = $r_4 \oplus r_1 = 1 \oplus 0 = 1$. Output-bit = $r_4 = 1$. Nieuw register: (1, 0, 0, 0).

De resulterende bitreeks na 8 stappen: 1, 0, 0, 1, 0, 0, 0, 1, ... Dit proces zet je nu voort voor de echte puzzel — maar met een register van lengte 29, tap-posities 29 en 2, en de seed uit Fase 1.

3.3 Van bitreeks naar matrices

Start het register met de 29-bits seed die je in Fase 1 hebt gevonden (het meest linkse bit is r_1). Genereer op de hierboven beschreven manier een reeks van minstens 1682 output-bits.

- De **eerste 841 bits** vul je, rij voor rij, in een rooster van 29×29 — dit wordt matrix A .
- De **volgende 841 bits** (bit 842 t.e.m. 1682) vul je op dezelfde manier in een tweede rooster van 29×29 — dit wordt de maskeermatrix M .

Zelfcontrole. Tel, ter controle, het totaal aantal enen in je matrices:

- Matrix A moet exact **421** enen bevatten (op 841 cellen).
- Matrix M moet exact **416** enen bevatten (op 841 cellen).

Klopt dit niet? Zoek dan eerst de fout in je bitreeks voor je verdergaat — een enkele omgewisselde bit vroeg in de reeks verschuift alles wat erna komt.

3.4 Invulrooster — matrix A

Vul hier, rij voor rij, de eerste 841 output-bits in (bit 1 bovenaan-links, verder rij per rij).

[illegible]

3.5 Invulrooster — maskeermatrix M

Vul hier, rij voor rij, bit 842 t.e.m. 1682 in.

[illegible]

4 Fase 3 — De vercijferde matrix demaskeren

Je krijgt hieronder niet matrix B zelf, maar een **vercijferde versie** B_{enc} . Deze is ontstaan door de echte matrix B bit voor bit te XOR-en met jouw maskeermatrix M :

$$B_{\text{enc}} = B \oplus M \pmod{2}.$$

Omdat XOR zijn eigen inverse is ($a \oplus b \oplus b = a$), geldt: zodra je M hebt (uit Fase 2), demaskeer je met exact dezelfde operatie:

$$B = B_{\text{enc}} \oplus M \pmod{2}.$$

Bereken dit cel voor cel (841 keer XOR) om de echte matrix B te bekomen.

De echte matrix B moet, ter controle, exact **459** enen bevatten.

De vercijferde matrix B_{enc}

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
1	0	0	1	1	0	0	1	0	1	0	0	0	0	1	1	0	1	1	1	1	0	1	0	1	1	0	0	0	0
2	1	0	0	1	1	1	0	1	1	1	0	0	0	0	0	0	0	0	1	0	1	1	0	0	1	0	0	1	0
3	0	0	1	0	1	1	0	0	0	0	1	0	1	0	1	1	1	1	0	1	0	0	1	0	0	1	0	1	1
4	0	0	0	1	1	0	1	0	1	0	0	1	0	0	1	1	0	1	1	1	0	0	0	0	0	0	1	1	0
5	1	0	1	0	0	0	1	0	1	0	1	0	0	0	0	1	1	1	0	1	0	0	0	1	1	1	1	0	0
6	1	1	0	0	1	1	0	0	0	0	1	1	0	1	1	0	1	1	0	1	0	1	1	1	1	0	0	0	1
7	0	1	1	0	1	1	0	0	0	0	1	0	0	1	0	1	0	1	1	0	0	1	0	0	1	1	1	0	1
8	0	0	0	1	0	1	1	1	1	0	1	0	0	0	0	0	1	1	1	0	0	1	0	1	1	0	1	1	0
9	1	1	1	1	0	1	0	1	1	1	0	0	1	0	1	0	0	0	1	0	1	0	0	1	0	0	0	0	1
10	0	1	0	1	1	1	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	1	0	1	1	1	0	0	1
11	1	0	1	0	0	1	0	0	0	0	1	1	0	1	1	1	1	0	0	1	1	1	0	0	0	0	1	0	1
12	1	0	0	0	0	1	1	0	0	1	0	0	0	1	1	1	0	1	0	1	1	1	0	0	1	1	0	1	0
13	1	0	1	0	1	1	1	1	0	0	0	1	1	0	1	0	0	1	0	1	0	0	1	1	1	1	0	0	0
14	0	1	1	1	1	0	0	1	0	1	0	1	0	0	0	0	1	0	1	1	0	1	1	1	0	1	0	1	1
15	0	0	0	0	0	0	1	1	1	1	1	0	1	1	0	1	1	1	0	0	0	1	0	0	0	0	1	0	1
16	1	1	1	1	0	1	1	0	1	1	1	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
17	0	1	0	0	1	0	1	1	0	0	0	1	0	1	1	1	1	1	1	0	1	1	1	1	0	0	1	0	0
18	1	0	0	0	0	1	0	1	1	1	0	0	0	0	1	1	0	1	1	1	1	0	1	1	0	1	0	0	1
19	1	1	0	0	1	0	0	1	0	1	0	0	0	1	1	0	1	0	0	0	1	0	1	1	0	0	1	1	1
20	0	0	1	1	1	0	1	1	0	1	1	0	0	1	0	1	0	0	1	0	1	0	0	1	0	0	1	0	0
21	1	0	1	0	0	0	1	1	0	1	1	1	0	1	1	0	0	1	1	0	1	0	1	1	1	0	1	1	1
22	1	1	1	0	1	1	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	1	1	1	0	1	1	0	0
23	1	0	0	0	0	1	1	1	1	1	1	1	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	1	1
24	0	0	1	0	0	1	0	1	0	0	1	1	1	0	1	1	1	0	0	0	0	0	0	1	0	1	1	1	1
25	1	0	0	1	1	1	1	0	1	1	0	1	1	0	0	1	0	1	1	0	0	0	1	0	0	1	0	1	0
26	0	0	0	0	0	1	0	1	1	1	0	0	0	1	1	1	0	1	1	0	1	1	1	0	1	0	1	0	1
27	0	1	0	1	1	0	0	0	0	0	1	0	1	0	0	0	1	0	0	0	1	0	0	0	1	1	1	0	0
28	0	0	0	0	0	1	0	1	0	0	1	0	0	0	1	1	1	0	0	0	0	0	1	0	0	0	0	1	1
29	1	0	1	1	0	1	1	1	1	1	1	0	0	1	1	0	0	0	1	0	1	1	0	0	1	0	1	1	0

Invulrooster — matrix B

Noteer hier, cel voor cel, het resultaat van $B = B_{\text{enc}} \oplus M \pmod{2}$.

[illegible]

5 Fase 4 — Gauss-Jordan-eliminatie modulo 2

Nu je zelf matrix A hebt opgebouwd (Fase 2) en matrix B hebt gedemaskeerd (Fase 3), is de rest identiek aan een klassiek stelsel: vind de unieke matrix X (opnieuw 29×29 , enkel 0'en en 1'en) die voldoet aan

$$A \cdot X \equiv B \pmod{2}.$$

Dit is de finale opgave: zodra je X volledig hebt uitgeschreven, is de opdracht voltooid.

Waarom dit werkt. Rekenen “modulo 2” betekent: elke optelling levert een 0 of een 1 op, nooit meer. Concreet is $1 + 1 = 0$, $1 + 0 = 1$, $0 + 0 = 0$ — dit is precies de logische XOR-operatie. Matrices met enkel 0'en en 1'en, waarbij alle rekenwerk op deze manier gebeurt, leven in wat wiskundigen het **eindig veld** $\text{GF}(2)$ noemen (Galois-veld met 2 elementen). Alle vertrouwde regels van lineaire algebra — matrixvermenigvuldiging, Gauss-eliminatie, inverteerbaarheid — blijven gewoon geldig, alleen de optelling verandert. Deze wiskunde is de basis van cryptografie en codeertheorie in het algemeen.

5.1 Het principe

Om X te vinden uit $AX \equiv B \pmod{2}$, bouw je de **uitgebreide matrix** $[A \mid B]$ (matrix A en matrix B naast elkaar geplakt, samen dus 29×58). Door rijoperaties toe te passen — steeds modulo 2 — herleid je het linkerdeel A tot de identiteitsmatrix I . Wat er dan rechts overblijft, is exact de gezochte matrix X :

$$[A \mid B] \xrightarrow{\text{rijoperaties mod 2}} [I \mid X].$$

Toegelaten rijoperaties:

- (i) Twee rijen verwisselen.
- (ii) Eén rij optellen (XOR) bij een andere rij.

Vermenigvuldigen van een rij met een constante is hier overbodig: de enige niet-triviale constante modulo 2 is 1 zelf, dus deze operatie doet niets.

5.2 Het algoritme, stap voor stap

1. Start bij kolom 1, rij 1.
2. Zoek in de huidige kolom (op of onder de huidige rij) een rij met een 1 in die kolom — dit wordt de **pivotrij**. Is er geen enkele 1 te vinden, ga naar de volgende kolom.
3. Verwissel indien nodig deze pivotrij met de huidige rij, zodat de pivot op de diagonaal staat.
4. Tel deze pivotrij (XOR) op bij *elke andere rij* die in deze kolom een 1 heeft — zowel erboven als eronder. Na deze stap staat er in de hele kolom nergens anders meer een 1 dan in de pivotrij zelf.
5. Ga naar de volgende rij en kolom, herhaal.
6. Na 29 succesvolle pivotstappen staat links I , en rechts staat X .

5.3 Volledig uitgewerkt minivoorbeeld (3×3)

Om de methode te illustreren lossen we eerst een kleine versie op. Stel:

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}.$$

Uitgebreide matrix:

$$[A \mid B] = \left(\begin{array}{ccc|ccc} 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 & 1 \end{array} \right)$$

Stap 1 — kolom 1. Rij 1 heeft al een 1 op de diagonaal (pivot gevonden, geen verwisseling nodig). Tel rij 1 op bij elke andere rij die in kolom 1 een 1 heeft: dat is rij 3.

$$R_3 \leftarrow R_3 + R_1 = (1+1, 0+1, 1+0 \mid 1+1, 1+0, 1+1) = (0, 1, 1 \mid 0, 1, 0)$$

Nieuwe matrix:

$$\left(\begin{array}{ccc|ccc} 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{array} \right)$$

Stap 2 — kolom 2. Rij 2 heeft een 1 op de diagonaal. Tel rij 2 op bij elke andere rij met een 1 in kolom 2: dat zijn rij 1 én rij 3.

$$R_1 \leftarrow R_1 + R_2 = (1+0, 1+1, 0+1 \mid 1+0, 0+1, 1+0) = (1, 0, 1 \mid 1, 1, 1)$$

$$R_3 \leftarrow R_3 + R_2 = (0+0, 1+1, 1+1 \mid 0+0, 1+1, 0+0) = (0, 0, 0 \mid 0, 0, 0)$$

Nieuwe matrix:

$$\left(\begin{array}{ccc|ccc} 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right)$$

Hier zien we dat rij 3 volledig nul is geworden in het linkerdeel — dit *specifieke* kleine voorbeeld heeft dus een niet-inverteerbare A (rang 2 in plaats van 3), en levert dus geen unieke oplossing. Dit minivoorbeeld illustreert enkel de rijoperaties zelf; in de echte puzzel hieronder is gegarandeerd dat A volledig inverteerbaar is (rang 29), dus daar loop je dit probleem niet tegen het lijf — elke kolom krijgt gegarandeerd een pivot.

Controleer je werk onderweg. Na elke pivotstap mag je gerust teruggaan en met de hand $A \cdot X \stackrel{?}{=} B$ verifiëren op een paar losse rijen — dat is veel sneller dan de hele eliminatie opnieuw te doen, en vangt rekenfouten vroeg op.

5.4 Werkblad — de uitgebreide matrix $[A \mid B]$ (29×58)

Dit is je volledige werkruimte: matrix A (links) en matrix B (rechts) naast elkaar, precies zoals in §5.1 beschreven. Voer hier je rijoperaties uit totdat het linkerdeel de identiteitsmatrix I is geworden — wat dan rechts overblijft, is X .

The figure displays two identical, empty 29x29 grids, labeled A and B, used for a 2D Ising model simulation. Each grid has a horizontal axis (columns) and a vertical axis (rows), both indexed from 1 to 29. The grids are separated by a vertical line, with grid A on the left and grid B on the right. The grids are intended for visualizing the spin configurations of the Ising model during a simulation.

Tip: bij 29 pivotstappen op een matrix van deze breedte is één werkblad meestal niet genoeg — gebruik gerust los papier om de matrix na elke stap of elke paar stappen opnieuw over te schrijven, en bewaar dit blad als startpositie.

Noteer hier, na de volledige Gauss-Jordan-eliminatie op de echte 29×29 -matrices A en B , de rechterhelft van de gereduceerde uitgebreide matrix — dit is je volledige eindresultaat X .

[illegible]